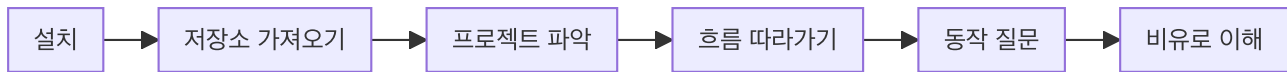


Claude Code: 코드베이스와 대화하기

박사홍 · sahong.pak@gmail.com

Road map



- I. Claude Code 설치하기 — 설치 · 환경 변수 · 처음 실행 · 로그인 · 모델 선택
- II. 전체 구조 파악하기 — 2048 저장소를 가져와 프롬프트 ①~④로 대화

I. Claude Code 설치하기

설치하기

PowerShell에서 한 줄 명령으로 설치

```
irm https://claude.ai/install.ps1 | iex
```

🔗 설치 결과 확인

- Claude Code successfully installed! 와 버전 표시
- 설치 위치 C:\Users\<사용자>\.local\bin\claude.exe
- Setup notes 의 PATH 안내는 오류가 아니라 다음 단계(환경 변수 등록) 안내

⚠️ Path 등록 확인

새 PowerShell 창에서 `claude --help` 가 실행되지 않으면 Path 등록 필요

```
PS C:\Users\SSAFY\Desktop> irm https://claude.ai/install.ps1 | iex
Setting up Claude Code...

✔ Claude Code successfully installed!

Version: 2.1.258

Location: C:\Users\SSAFY\.local\bin\claude.exe

Next: Run claude --help to get started

⚠ Setup notes:
• Native installation exists but C:\Users\SSAFY\.local\bin is not in your PATH. Add it by opening:
  System Properties → Environment Variables → Edit User PATH → New → Add the path above. Then
  restart your terminal.

✔ Installation complete!
```

환경 변수 등록

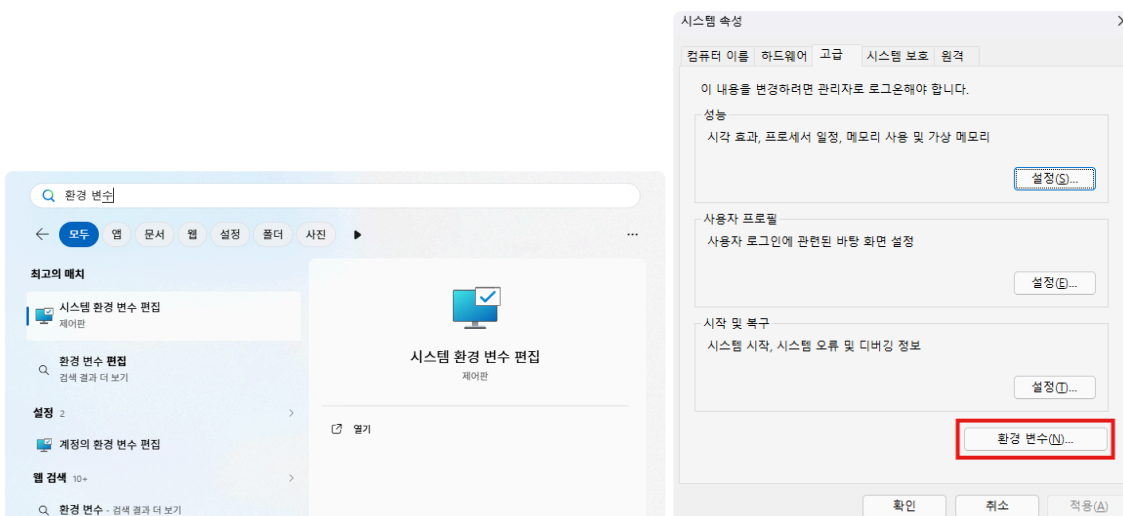
설치 경로를 Path에 추가해야 어느 폴더에서든 claude 명령 실행 가능

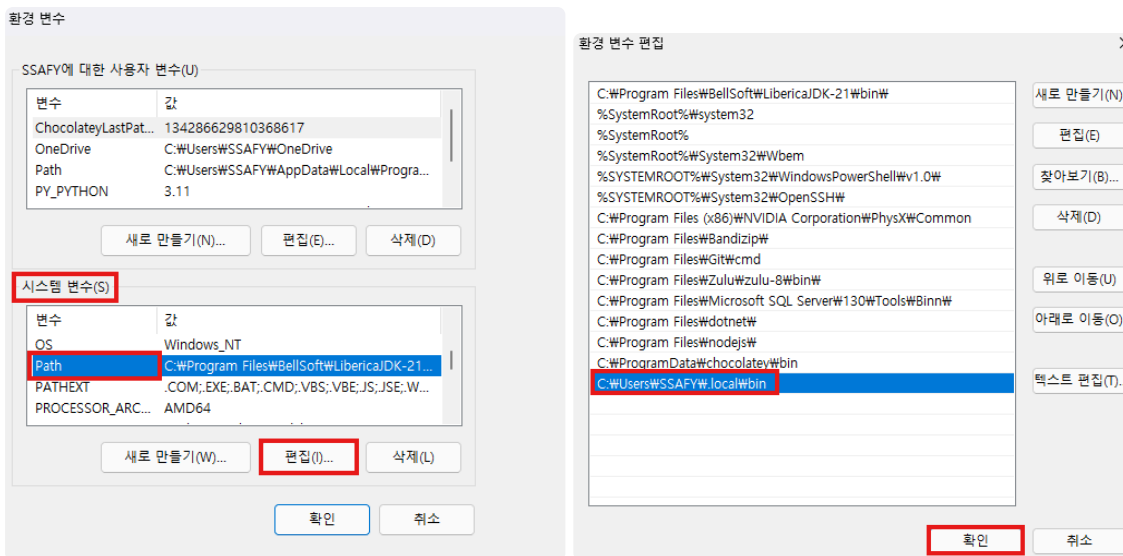
📖 순서

1. 검색창에 '환경 변수' 검색 → 시스템 환경 변수 편집
2. 시스템 속성 > 고급 탭 > 환경 변수(N)
3. 시스템 변수에서 **Path** 선택 → 편집(I)
4. 새로 만들기 → C:\Users\<사용자>\.local\bin 입력 → 확인

⚠ 새 창에서 확인

적용 후 PowerShell 창을 새로 열어야 변경된 Path 반영





처음 실행하기

PowerShell에서 `claude` 를 입력하면 테마 선택과 로그인 순서로 진행

① 순서

1. 테마 선택 (`/theme` 으로 변경 가능)
2. 로그인 방식 선택 → **1. Claude account with subscription**
3. 브라우저 로그인 → `Login successful` 확인 → Enter
4. 작업 폴더 접근 확인 → **Yes, I trust this folder**

② ‘이 폴더를 신뢰합니까?’ 는 무슨 뜻일까?

Claude Code가 이 폴더의 파일을 읽고 · 수정하고 · 실행할 수 있게 허용한다는 뜻
내가 만든 프로젝트나 잘 아는 오픈소스라면 Yes
출처가 불분명한 폴더라면 먼저 내용을 살펴본 뒤 결정

모델 선택

`/model` 로 상황에 맞는 모델 선택 (처음에는 Default 그대로 두어도 충분)

모델	이런 때
Opus 5 (Default)	일상적인 작업부터 복잡한 작업까지 기본값
Fable 5.1	가장 어렵고 오래 걸리는 작업
Sonnet 5	반복적인 일상 작업을 효율적으로
Haiku 4.5	빠른 답변이 필요할 때

- 프롬프트 창에 `/model` 입력 → 번호로 선택

- 모델 이름 옆 설명으로 각 모델의 용도 확인

II. 전체 구조 파악하기

협업, 어디서 막히나요?

② 팀 프로젝트에서 어떤 어려움을 겪었나요?

- 팀원이 짰 코드를 열어도 무엇을 하는지 모름
- 내 파트가 아니면(FE ↔ BE) 남의 일처럼 느껴짐
- 합칠 때 처음 보는 파일이 한꺼번에 쏟아짐
- 질문하기 미안해서 혼자 헤멤
- 발표 전날, 전체 구조를 아무도 설명하지 못함

③ 상대방의 코드, 쉽게 이해했나요?

읽는 데 시간이 오래 걸리고, 읽어도 왜 그렇게 짰는지는 모름
결국 만든 사람에게 묻거나, 안 읽고 내 파트만 진행

코드를 이해한다는 것

세 가지 질문에 답할 수 있으면 남의 코드도 내 코드처럼 다룰 수 있음

질문	막히는 이유	오늘의 프롬프트
무엇을 하는가 — 구조	파일이 많아 어디부터 볼지 모름	① 구조 파악
어떻게 동작하는가 — 흐름	함수가 여기저기 흩어져 흐름이 눈에 안 보임	② 흐름 따라가기
왜 이렇게 만들었는가 — 의도	물어볼 사람이 없거나 이미 퇴근	③ 동작 질문 · ④ 비유

④ 실습 대상

실습 대상은 내가 만들지 않은 오픈소스 **2048** — 팀원 코드를 처음 여는 상황과 같음

실습 저장소 가져오기

GitHub의 2048 게임 저장소를 내 컴퓨터로 복제(clone)

① 순서

1. GitHub ▶ Code ▶ HTTPS 주소 복사 —
`https://github.com/gabrielecirulli/2048.git`
2. PowerShell에서 `git clone` 실행

```
PS C:\Users\SSAFY\Desktop> git clone  
https://github.com/gabrielecirulli/2048.git  
Cloning into '2048'...  
remote: Enumerating objects: 1318, done.  
Receiving objects: 100% (1318/1318), 624.25 KiB | 31.21 MiB/s, done.  
Resolving deltas: 100% (767/767), done.
```

바탕화면(Desktop)에 2048 폴더가 생기면 완료

프로젝트 폴더에서 시작하기

실행한 폴더 = 작업 공간(workspace)

```
PS C:\Users\SSAFY\Desktop> cd .\2048\  
PS C:\Users\SSAFY\Desktop\2048> claude
```

❓ 왜 프로젝트 폴더 안에서 실행해야 할까?

실행한 폴더의 파일만 읽기 가능
폴더 신뢰를 마치면 그 코드베이스에 대해 질문 가능

프롬프트 ① 프로젝트 파악하기

무엇을 하는 프로젝트인지, 각 파일이 어떤 역할인지 질문

> 이 프로젝트가 뭘 하는 프로젝트인지, 각 파일이 어떤 역할인지 프로그래밍을 잘 모르는 사람도 이해할 수 있게 설명해줘.

- Reading `js\application.js`
 - └ `js\application.js`

✍ 프롬프트에 담긴 세 가지

- 무엇을 — 이 프로젝트가 뭘 하는지 · 목적부터 질문
- 어떻게 — 각 파일이 어떤 역할인지 · 구조를 파일 단위로 요청
- 누구에게 — 프로그래밍을 잘 모르는 사람도 · 대상 독자를 지정하면 설명의 깊이가 달라짐

결과 ① 프로젝트 소개와 전체 구조

Claude의 답 — 유명한 숫자 퍼즐 게임 “2048”의 원본 소스 코드 (Gabriele Cirulli)

- 4×4 판에서 방향키(↑↓←→)로 타일을 밀면 같은 숫자 타일 두 개가 만나 합쳐지며 두 배로 증가 (2+2=4, 4+4=8 ...)
- 숫자를 계속 합쳐서 2048 타일을 만드는 것이 목표

폴더 / 파일	역할
index.html	게임 화면 뼈대 (제목, 점수판, 게임판, 안내문구)
style/	디자인 (색깔, 애니메이션, 반응형 레이아웃)
js/	게임 동작 로직 — 실제 ‘두뇌’ 역할
meta/	아이폰 홈화면 아이콘 등 부가 이미지
favicon.ico	브라우저 탭에 표시되는 작은 아이콘

- 기타 파일 — README.md 프로젝트 소개 · CONTRIBUTING.md 기여 규칙 · LICENSE.txt MIT 라이선스 · Rakefile scss → css 빌드 · .jshinttrc / .gitignore 개발 도구 설정
- style/ 폴더 — main.scss 원본 → main.css 브라우저가 쓰는 변환본 · helpers.scss 반복 스타일 재사용 · fonts/ 글꼴

결과 ② js/ 폴더 — 각 파일의 역할

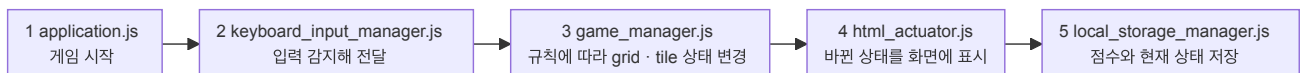
“게임을 사람의 몸에 비유하면 이렇게 나눌 수 있어요”

파일	역할	비유
application.js	게임을 실제로 시작시키는 시작점	전원 버튼
game_manager.js	게임의 전체 규칙 총괄 (타일 이동, 합치기, 승리/패배 판정)	게임의 두뇌 · 심판
grid.js	4×4 격자판 자체를 관리 (어느 칸이 비었는지, 타일이 어디 있는지)	게임판
tile.js	타일 하나(숫자 하나)의 정보 (값, 위치)	말(패) 한 개
html_actuator.js	게임 상태를 실제 화면 그림으로 그려주는 역할	화면에 그림 그려주는 손
keyboard_input_manager.js	키보드 방향키, 스와이프(모바일 터치), 버튼 클릭을 감지	귀 · 손 (입력 감지)
local_storage_manager.js	점수와 진행 상태를 브라우저에 저장해 새로 고침해도 이어할 수 있게 함	기억 저장소
*_polyfill.js (3개)	구형 브라우저에서도 게임이 돌아가게 해주는 호환성 보조 코드	통역사

✎ 비유 열이 생긴 이유

‘비유’ 열은 요청하지 않았는데도 Claude가 덧붙인 것 — “프로그래밍을 잘 모르는 사람도 이해할 수 있게” 라는 조건이 만든 차이

결과 ③ 한 판이 진행되는 흐름



✎ 한 줄 요약

“판(grid)과 타일(tile)로 상태를 관리하고, 입력(input)을 받아 규칙(game_manager)대로 바꾼 다음, 화면(actuator)에 그려주고 저장(storage)까지 해주는 웹 기반 숫자 퍼즐 게임”

- 파일 하나 열어보지 않고 프로젝트의 목적 · 파일별 역할 · 한 판의 진행 순서를 파악
- 다음은 실제 코드가 그 흐름을 어떻게 만드는지, 키 입력 한 번을 끝까지 추적

프롬프트 ② 동작 흐름 따라가기

키를 누른 순간부터 화면이 바뀔 때까지, 코드에서 일어나는 일을 순서대로 질문

> 내가 방향키를 누르면 타일이 움직이고 합쳐지잖아. 키를 누른 순간부터 화면이 바뀔 때까지 코드에서 무슨 일이 일어나는지, 택배가 배송되는 과정 설명하듯 순서대로 알려줘. 직관적으로 파악할 수 있도록 HTML로 표현해주면 좋을 것 같아.

- Reading `js\keyboard_input_manager.js`, `js\game_manager.js`, `js\html_actuator.js` ...
- Wrote `2048\tile-delivery-tracker.html`

프롬프트 포인트

- 시작과 끝 — 키를 누른 순간 → 화면이 바뀔 때까지 · 추적 구간을 잘라 주면 답이 흐트러지지 않음
- 익숙한 비유 — 택배가 배송되는 과정처럼 · 낯선 코드 흐름을 아는 이야기에 얹어 이해
- 출력 형식 — HTML로 표현해줘 · 파일로 받아 브라우저에서 열어보며 설명 가능

결과 — 타일 배송 추적기 (HTML)

방향키 한 번 = 배송 한 건. 송장(키 입력 · 트리거 · 수단 · 시간)을 먼저 표시하고, 정류장마다 담당 파일과 코드 조각을 함께 표시

#	단계	담당 파일 · 함수
1	주문 접수 — 방향키 입력	<code>keyboard_input_manager.js</code>
2	분류 — 어느 쪽부터 처리할지 결정	<code>game_manager.js</code> · <code>move()</code> , <code>buildTraversals()</code>
3	이동 & 병합 — 같은 상품은 한 박스로	<code>game_manager.js</code> · <code>findFarthestPosition()</code> , <code>move()</code>
4	신규 입고 — 빈 창고에 상품 한 개	<code>game_manager.js</code> · <code>addRandomTile()</code>
5	기록 — 현황판과 최고 기록 갱신	<code>local_storage_manager.js</code> · <code>actuate()</code>
6	하차 — 화면이라는 진열대에 내려놓기	<code>html_actuator.js</code> · <code>actuate()</code> , <code>addTile()</code>
7	수령 확인 — 눈으로 보는 부드러운 이동	<code>main.scss</code> · <code>.tile transition</code>

- 결과물: [007_동작 흐름 시각화.html](#)

프롬프트 ③ 동작 질문하기

직접 플레이하며 관찰한 현상을 근거로 ‘어떻게 막는지 · 왜 그렇게 만들었는지’ 질문

> 게임을 해보니깐 2-2-4가 나란히 있을 때 밀면 8이 되지 않고 4-4로 끝나던데, 한 번의 이동에서 같은 타일이

두 번 합쳐지지 않게 코드가 어떻게 막고 있어? 왜 이렇게 만들었을까?

- 좋은 관찰이에요. 코드에서 막는 지점은 딱 두 군데예요.

✍ ① 각 칸은 한 번의 이동에서 딱 한 번만 검사

`move()` 는 이중 `forEach`로 모든 칸을 한 번씩만 통과
되돌아가서 다시 검사하는 루프 없음

```
traversals.x.forEach(function (x) {  
  traversals.y.forEach(function (y) {  
    tile = self.grid.cellContent({x:x, y:y});  
    if (tile) { /* 이동 or 병합 */ }  
  });  
});
```

✍ ② 이미 합쳐진 타일엔 다시 합치기 불가

`!next.mergedFrom` — 명시적 잠금장치
“상대가 이번 이동에서 이미 합쳐진 결과물이면 안 돼”

```
if (next && next.value === tile.value  
    && !next.mergedFrom) {  
  var merged = new Tile(positions.next,  
                        tile.value * 2);  
  merged.mergedFrom = [tile, next];  
}
```

2, 2, 4 를 오른쪽으로 밀면

`buildTraversals` 는 오른쪽 이동일 때 벽에 가까운 칸부터 처리 `[3, 2, 1, 0]` — 8이 아니라 4,
4 로 종료

순서	처리 대상	결과
1	이미 벽쪽에 있던 4	그대로 벽에 붙음 (4)
2	그 왼쪽 2	앞의 4와 값이 달라 병합 안 됨, 한 칸만 이동
3	맨 왼쪽 2	방금 옮겨온 2와 만나 병합 → 4 생성

- 이 시점에서 루프 종료
- 새로 만들어진 4가 옆의 4와 같아졌지만 그 칸은 이미 지나간 칸이라 다시 검사하지 않음
- !next.mergedFrom 은 2, 2, 2 처럼 같은 값이 셋일 때 세 번째 타일이 방금 만든 병합 타일을 또 먹는 걸 방지

❓ 왜 이렇게 만들었을까?

- 한 번의 입력 = 예측 가능한 결과 — 연쇄 병합을 허용하면 한 번의 입력으로 보드가 도미노처럼 정리되어 ‘어디까지 합쳐질지 예측하기’라는 재미가 무너짐
- 점수 · 난이도 균형 — 한 번의 키 입력으로 점수가 기하급수적으로 튀거나 보드가 갑자기 비어, 원작(1024/Threes 계열)이 의도한 ‘천천히 채우고 신중하게 합치는’ 긴장감이 사라짐
- 애니메이션 · 인지 부담 — 여러 단계의 연쇄 합체를 한 번에 보여주면 뭐가 뭐랑 합쳐졌는지 따라가기 어려움 · ‘한 번 밀면 최대 한 단계’ 규칙으로 화면·멘탈 모델 단순화

프롬프트 ④ 비전공자 수준으로 이해하기

코드를 한 줄도 못 읽는 사람에게, 전체 구조를 하나의 비유로 설명하고 시각화해 저장하도록 요청

> 코드를 한 줄도 못 읽는 사람에게 설명한다고 생각하고, 이 게임의 코드의 전체 구조를 하나의 비유로 설명해줘.

가능하다면 시각화해서 저장해줘.

- Skill(artifact-diagramming) Successfully loaded skill
- Artifact(...\scratchpad\tile-diner-floorplan.html) Published 
- 로컬 파일로도 저장했습니다: C:\Users\SSAFY\Desktop\2048\tile-diner-floorplan.html

Claude가 한 일

작성 요령(Skill)을 스스로 불러오고 → 웹 페이지(Artifact)로 발행 → 프로젝트 폴더에도 HTML로 저장

결과 — 타일식당 평면도 (HTML)

Claude가 고른 비유 “작은 식당” — 손님 → 웨이터 → 주방장 → 플레이팅 → 다이닝룸

방	파일	하는 일
개점 준비	application.js	손님이 오기 전 셔터를 올려 개점
손님	—	방향키 하나를 눌러 ‘이 방향으로 밀어 주세요’ 주문
웨이터	keyboard_input_manager.js	주문 방향을 확인해 주방에 전달
주방장	game_manager.js (조리대 grid.js · 재료 tile.js)	어느 재료부터 밀지 정하고, 같은 재료 둘이 만나면 하나로 합쳐 점수를 매김
플레이팅	html_actuator.js	완성된 결과를 접시(화면) 위에 올림
다이닝룸	—	손님이 결과를 보고 다음 주문을 정하는 자리
장부실	local_storage_manager.js	점수와 최고 기록을 공책에 적어 새로고침해도 보관

- 결과물: [010_비전공자 수준으로 이해하기 시각화.html](#)

정리

코드베이스와 대화하는 네 가지 프롬프트

단계	프롬프트	얻은 것
① 구조 파악	“무엇을 하는 프로젝트인지, 각 파일의 역할을 비전공자 눈높이로 설명해줘”	폴더 · 파일 역할 표 + 비유
② 흐름 따라가기	“키 입력부터 화면 변화까지, 택배 배송처럼 순서대로, HTML로”	7단계 타일 배송 추적기
③ 동작 질문	“직접 플레이하며 관찰한 현상을 근거로 ‘어떻게 막고, 왜 그렇게 만들었는지’”	코드 근거 두 곳 + 설계 의도
④ 비유로 시각화	“코드를 못 읽는 사람에게 하나의 비유로, 시각화해서 저장”	타일식당 평면도 (HTML)

네 프롬프트에 공통으로 들어간 것

대상 독자 지정 · 익숙한 비유 요청 · 출력 형식 지정 · 코드 근거 요구

AI의 답변, 어떻게 검증할까?

할루시네이션은 결함이 아니라 특성 — 틀린 내용도 자신 있게 말하므로 검증은 사용자의 몫

	① 근거 요구 — AI에게	② 직접 재현 — 내가	③ 교차 확인 — 다른 눈
방법	파일 · 줄 번호 · 코드 조각을 함께 요구 / 추측한 부분은 ‘추정’ 표시 요구	설명한 동작을 직접 실행 (2-2-4 밀어 보기 → 4-4) / 말한 함수를 열어 코드와 대조	같은 질문을 다른 말로 다시 / 새 세션 · 다른 모델에 재질문 / 작성자 · README와 대조
예시	“어느 파일 몇 번째 줄인지 코드 조각과 함께 보여줘”	!next.mergedFrom 이 실제로 game_manager.js 에 있는지 확인	질문을 바꿔 물었을 때 답이 달라지면 의심

⚠ 위험 신호 — 이럴 때는 멈추고 검증

존재하지 않는 파일 · 함수 이름 · 실행해 보면 다른 결과 · 근거 없이 매끄러운 설명 · 한 번도 파일을 읽지 않고 답함

📌 오늘 실습에서의 예

오늘 프롬프트 ③이 그 예 — ‘어떻게 막고 있어?’에 코드 두 군데를 근거로 답했고, 게임에서 직접 재현해 확인

팀 프로젝트에서 활용하기

1학기 · 2학기 팀 프로젝트 모두 같은 흐름 — 인원만 다를 뿐, 처음 보는 코드는 늘 존재

단계	상황	프롬프트 예시	오늘 배운 것
프로젝트 시작	역할 분담 · 템플릿 파악	“이 템플릿 프로젝트의 폴더 구조와 각 파일 역할을 설명해줘” / “참고하려는 오픈소스를 클론했어. 무엇을 하는 프로젝트인지 알려줘”	① 구조 파악
기능 개발	다른 파트 코드 이해	“로그인 버튼 클릭부터 DB 저장까지 무슨 일이 일어나는지 순서대로” / “백엔드 API 응답이 화면에 어떻게 그려지는지 흐름을 따라가 줘”	② 흐름 따라가기
통합 · 디버깅	버그 원인 · 코드 리뷰	“새로고침하면 로그인이 풀리는데 어디서, 왜 그렇게 되는지 설명해줘” / “이번 PR이 바꾼 동작을 리뷰어 입장에서 요약해줘”	③ 동작 질문
발표 · 문서	아키텍처 설명 · README	“우리 서비스 구조를 심사위원도 이해하게 하나의 비유로 시각화해줘” / “README에 넣을 아키텍처 설명을 비전공자 눈높이로 써줘”	④ 비유로 시각화

① 실행 순서

프로젝트 폴더에서 `claude` 실행 → 폴더 신뢰 → 질문

⚠ 비밀 정보 주의

`.env` 같은 비밀 정보 파일은 대화에 올리지 않기

처음 보는 코드베이스가 생길 때마다 팀원 모두 ① → ② → ③ → ④ 순서로 대화 → 팀의 이해 속도가 맞춰짐

부록

프롬프트 모음 (복사해서 사용)

이 프로젝트가 뭘 하는 프로젝트인지, 각 파일이 어떤 역할인지 프로그래밍을 잘 모르는 사람도 이해할 수 있게 설명해줘.

내가 방향키를 누르면 타일이 움직이고 합쳐지잖아. 키를 누른 순간부터 화면이 바뀔 때까지 코드에서 무슨 일이 일어나는지, 택배가 배송되는 과정 설명하듯 순서대로 알려줘. 직관적으로 파악할 수 있도록 HTML로 표현해주면 좋을 것 같아.

게임을 해보니까 2-2-4가 나란히 있을 때 밀면 8이 되지 않고 4-4로 끝나던데, 한 번의 이동에서 같은 타일이 두 번 합쳐지지 않게 코드가 어떻게 막고 있어? 왜 이렇게 만들었을까?

코드를 한 줄도 못 읽는 사람에게 설명한다고 생각하고, 이 게임의 코드의 전체 구조를 하나의 비유로 설명해줘. 가능하다면 시각화해서 저장해줘.

실습 자료

- 실습 저장소: <https://github.com/gabrielecirulli/2048>
- `resource/001_Claude Code Install/` — 설치 · 환경 변수 · 로그인 · 모델 선택 캡처
- `resource/002_전체 구조 파악하기/` — 프롬프트 캡처, 결과 텍스트, 타일 배송 추적기 · 타일식당 평면도 HTML